EduStack Masterclass

Your Ultimate Computer Science & Engineering Hub

VOLUME 4 -- System Design, OS, Computer Networks & AI Microservice

Product-Based Company Interview Guide (Amazon, Microsoft, Visa, Oracle, JPMC)

OS Concepts | TCP/IP Networking | CAP Theorem | 20 FAANG Scenarios | Interview Playbook

Developer:	Shubham Kumar | CSE Student | NIT Patna
Target:	System Design Architect, Senior SDE, AI Infrastructure Engineer
Microservice:	Python 3.11 + FastAPI + Uvicorn ASGI Server
AI Models:	Google Gemini 1.5/2.0 Flash + LightRAG Knowledge Engine
Networks:	TCP/IP 4-Layer, HTTP/2 Multiplexing, TLS 1.3 Handshake
System Design:	CAP Theorem, Consistent Hashing, Rate Limiting, Sharding
Volume 4:	OS, Networks, System Design, 20 FAANG Scenarios, Playbook

github.com/ShubhamKumar968/EduStack--Your-Ultimate-Computer-Science-Hub
Volume 4 of 4 -- For Product-Based Company Interview Preparation

Table of Contents - Volume 4

SECTION 1

Operating System Core Concepts

Processes vs Threads, Context Switching, Deadlock conditions, Memory virtualization

1.1 Processes vs Threads

Understanding operating system abstractions is critical for backend engineering interviews.

Dimension	OS Process	OS Thread
Memory Space	Isolated virtual address space per process	Shared address space within parent process
Context Switch Cost	High - flush CPU Translation Lookaside Buffer (TLB)	Low - register reload only, TLB intact
IPC Requirement	Requires Inter-Process Communication (IPC, Pipes, Sockets)	Direct memory access via shared variables
Node.js Context	Node.js app runs as 1 main OS process	libuv thread pool maintains background worker threads

Deadlock - 4 Coffman Conditions & Prevention

- **Mutual Exclusion**: At least one resource must be held in a non-shareable mode.
- **Hold and Wait**: A process holds a resource while requesting additional resources.
- **No Preemption**: Resources cannot be forcibly taken from a process.
- **Circular Wait**: A closed chain of processes exists where each waits for a resource held by the next.
- **Prevention**: Banker's Algorithm for deadlock avoidance; strict lock ordering rule for thread synchronization.

SECTION 2

Computer Networks & Web Protocols

OSI 7 Layers, TCP 3-way handshake, HTTP/1.1 vs HTTP/2 vs HTTP/3, TLS 1.3

2.1 TCP 3-Way Handshake vs 4-Way Teardown

- **SYN**: Client sends SYN packet (seq = x) to server.
- **SYN-ACK**: Server responds with SYN-ACK packet (seq = y, ack = x + 1).
- **ACK**: Client responds with ACK packet (ack = y + 1). Connection ESTABLISHED.
- **TLS 1.3 Handshake**: Follows TCP handshake in 1 RTT, exchanging Diffie-Hellman keys.

Protocol	Transport Layer	Key Feature	EduStack Usage
HTTP/1.1	TCP	Head-of-Line Blocking, 1 request per TCP socket	Legacy client connections
HTTP/2	TCP	Binary framing, Multiplexing over 1 TCP socket, Header Compression (HPACK)	Production API communication
HTTP/3	UDP (QUIC)	Zero-RTT handshake, no TCP Head-of-Line blocking, connection migration	Future edge CDN routing

SECTION 3

System Design Framework & Trade-offs

CAP Theorem, BASE vs ACID, Consistency Models, Load Balancing Strategies

3.1 CAP Theorem Framework

In distributed systems, a database can guarantee at most TWO of the three CAP properties simultaneously during network partition:

- **Consistency (C)**: Every read receives the most recent write or an error.
- **Availability (A)**: Every non-failing node returns a non-error response without guaranteeing latest data.
- **Partition Tolerance (P)**: System continues operating despite network packet loss or node isolation.
- **MongoDB Atlas (CP)**: Prioritizes consistency and partition tolerance. During primary partition, secondary elections take ~10-12s (writes paused).

SECTION 4

Scalability & High Availability Patterns

Horizontal vs Vertical scaling, Database Sharding, Consistent Hashing algorithm

4.1 Consistent Hashing Algorithm

Consistent Hashing maps both servers and keys to a 360-degree virtual ring using a hash function (MD5/SHA-256). When a server node is added or removed, only K/N keys need remapping (where K = total keys, N = total nodes), preventing cache storm failures.

SECTION 5

Top 15 DSA Algorithmic Interview Patterns

Essential algorithmic problem-solving patterns for FAANG interviews

Pattern	Use Case	Example Problem
Sliding Window	Subarrays/Substrings with target criteria	Longest Substring Without Repeating Characters
Two Pointers	Sorted arrays/strings comparison	Container With Most Water, 3Sum
Fast & Slow Pointers	Cycle detection in linked lists/arrays	LinkedList Cycle II, Happy Number
BFS / Graph Traversal	Shortest path in unweighted graph	Word Ladder, Binary Tree Level Order Traversal
Dynamic Programming	Overlapping subproblems & optimal substructure	Coin Change, Longest Common Subsequence

SECTION 6

Python FastAPI ML Microservice Architecture

FastAPI Uvicorn server, LightRAG knowledge store, Gemini fallback loop

Python / System Design — EduStack Production

```python
# ml_services/main.py - Python FastAPI ML Microservice
from fastapi import FastAPI, HTTPException
import google.generativeai as genai

app = FastAPI(title="EduStack AI Microservice")

@app.post("/api/rag/generate-pyq")
async def generate_pyq(payload: dict):
    try:
        model = genai.GenerativeModel("gemini-1.5-flash")
        response = await model.generate_content_async(payload.get("prompt"))
        return {"success": True, "data": response.text}
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

SECTION 7

20 FAANG System Design & Incident Scenarios

Real-world system design interview questions and incident response blueprints

Q: Scenario 1: How do you handle 100,000 concurrent payment webhooks without dropping requests?

Answer:

Buffer webhooks in a message queue (AWS SQS or Redis Streams) -> Worker pool consumes messages at controlled rate -> Idempotency check via razorpayOrderId before DB write -> Return 200 OK to Razorpay immediately upon queue push.

-> Queue decouples ingestion from database processing.

-> Idempotency key prevents double-crediting users.

Q: Scenario 2: How do you prevent Botnet DDoS attacks on authentication routes?

Answer:

Deploy Cloudflare WAF at edge -> Enforce CAPTCHA after 3 failed login attempts -> Apply IP + User-Agent rate limiting via Redis -> Block IP ranges exhibiting bot signature headers.

-> Edge WAF filters bad traffic before hitting origin server.

-> Redis rate limiter tracks IP + fingerprint.

Q: Scenario 3: How do you execute zero-downtime MongoDB schema migrations?

Answer:

Expand-Contract pattern: (1) Add new optional fields to schema. (2) Deploy app version supporting both old and new fields. (3) Run background script migrating existing docs. (4) Remove old field code.

-> Expand-Contract pattern avoids breaking API compatibility during deploys.

Q: Scenario 4: How do you prevent Memory Exhaustion (OOM) when serving 100MB PDF files?

Answer:

Use Node.js Stream pipeline: fs.createReadStream(pdfPath).pipe(res). Streaming chunks (64KB) prevents loading 100MB buffer into V8 heap RAM.

-> Streams keep RAM usage constant (~64KB per active download).

Q: Scenario 5: How do you design a high-performance Leaderboard for 1M active users?

Answer:

Use Redis Sorted Sets (ZADD, ZREVRANGE). O(log N) insertion and score updates. Read top 100 users in O(log N + M) time directly from RAM.

-> Redis Sorted Sets (ZSET) provide O(log N) performance for live leaderboards.

Q: Scenario 6: How do you maintain session state across 10 horizontally scaled server instances?

Answer:

Centralize session storage in Redis cluster (express-session-redis adapter) or use stateless JWT httpOnly cookies.

-> Stateless JWT cookies or external Redis session cluster.

Q: Scenario 7: What strategy prevents Thundering Herd problem when cache expires?

Answer:

Mutex lock / Probabilistic Early Expiration (XFetch): First process acquiring lock refreshes cache; other requests serve stale cached data until refresh completes.

-> Cache locks prevent DB overload on cache miss bursts.

Q: Scenario 8: How do you handle MongoDB Primary node failure in production?

Answer:

MongoDB Replica Set automatically triggers election (~10-12 seconds). Mongoose client auto-reconnects to newly elected Primary node.

-> Replica Set election promotes secondary to primary automatically.

Q: Scenario 9: How do you secure internal communication between Node.js and Python microservice?

Answer:

mTLS (Mutual TLS) certificates or private virtual network isolation (Render.com internal network) with shared HMAC secret header.

-> Private network routing + mTLS certificate authentication.

Q: Scenario 10: How do you design an Analytics Event Collection pipeline for 10M events/day?

Answer:

Batch events in client memory -> Send POST payload every 10s -> Kafka/Kinesis stream ingestion -> ClickHouse or BigQuery data warehouse.

-> Kafka streaming ingestion to ClickHouse column-store database.

Q: Scenario 11: How do you design a URL Shortener like Bitly at scale?

Answer:

Base62 encoding of auto-incrementing ID or MD5 hash prefix -> Store mapping in NoSQL DB (MongoDB/Cassandra) -> Cache hot URLs in Redis.

-> Base62 encoding + Redis cache for O(1) redirection lookup.

Q: Scenario 12: How do you implement a Distributed Rate Limiter?

Answer:

Sliding Window Counter algorithm in Redis using Lua script (`EVAL`) for atomic execution across distributed instances.

-> Redis + Lua script for atomic sliding window rate limiting.

Q: Scenario 13: How do you prevent Race Conditions during Flash Sale inventory checkout?

Answer:

Redis atomic `DECR` command for inventory count OR MongoDB optimistic concurrency control using version key (`__v`).

-> Atomic Redis DECR or optimistic locking with Mongoose __v.

Q: Scenario 14: How do you design a Real-Time Notification System for millions of users?

Answer:

WebSockets for active connections -> Fallback to Server-Sent Events (SSE) -> Pub/Sub message broker (Redis Pub/Sub or RabbitMQ) for broadcasting.

-> WebSockets + Redis Pub/Sub for real-time notification fan-out.

Q: Scenario 15: How do you handle Database Connection Pool Exhaustion?

Answer:

Implement Circuit Breaker pattern (Opossum npm) -> Tune connection pool size -> Add read replicas for query offloading.

-> Circuit Breaker + Connection pool tuning + Read replicas.

Q: Scenario 16: How do you design a File Storage Service like AWS S3?

Answer:

Chunked multipart upload -> Metadata stored in MongoDB -> Binary chunks written to distributed object store (Ceph/MinIO) -> Edge CDN.

-> Multipart chunked upload + Metadata DB + Edge CDN caching.

Q: Scenario 17: How do you generate Globally Unique IDs in a distributed system?

Answer:

Twitter Snowflake ID algorithm: 64-bit integer (Timestamp 41 bits + Datacenter ID 5 bits + Worker ID 5 bits + Sequence number 12 bits).

-> Twitter Snowflake ID algorithm for k-ordered 64-bit unique IDs.

Q: Scenario 18: How do you implement Graceful Shutdown in Node.js?

Answer:

Listen for SIGTERM/SIGINT -> `server.close()` to stop accepting new requests -> Drain in-flight requests -> Close DB connection pool -> `process.exit(0)`.

-> SIGTERM listener drains in-flight requests and closes DB pools gracefully.

Q: Scenario 19: How do you design Multi-Region Data Replication for low latency?

Answer:

Active-Active multi-region deployment with GeoDNS routing -> DynamoDB Global Tables or MongoDB Global Clusters -> Asynchronous cross-region replication.

-> GeoDNS routing + Multi-Region active-active replication.

Q: Scenario 20: How do you design a Real-Time Collaborative Document Editor like Google Docs?

Answer:

Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs) over WebSockets with central authority server.

-> CRDTs / Operational Transformation over WebSockets.

SECTION 8

Executive Technical Interview Playbook

2-minute elevator pitch, behavioral STAR framework, system architecture defense cheat sheet

8.1 The 2-Minute Architecture Elevator Pitch

"EduStack is a high-performance computer science learning platform architected as a hybrid Node.js Express monolith and Python FastAPI microservice. I engineered a stateless authentication pipeline using JWTs in httpOnly, Secure, SameSite cookies with bcrypt-12 password hashing and Google OAuth 2.0. The database layer uses MongoDB Atlas with WiredTiger engine, custom compound indexes, and TTL auto-expiring OTP documents. The system integrates Razorpay payments with HMAC-SHA256 constant-time verification and zero-disk Cloudinary media streaming."